

Code Pretraining Improves Entity Tracking Abilities of Language Models

Najoung Kim*

Department of Linguistics
Boston University
najoung@bu.edu

Sebastian Schuster*

Department of Linguistics
University College London
s.schuster@ucl.ac.uk

Shubham Toshniwal*

NVIDIA
stoshniwal@nvidia.com

Abstract

Recent work has provided indirect evidence that pretraining language models on code improves the ability of models to track state changes of discourse entities expressed in natural language. In this work, we systematically test this claim by comparing pairs of language models on their entity tracking performance. Critically, the pairs consist of base models and models trained on top of these base models with additional code data. We extend this analysis to additionally examine the effect of math training, another highly structured data type, and alignment tuning, an important step for enhancing the usability of models. We find clear evidence that models additionally trained on large amounts of code outperform the base models. On the other hand, we find no consistent benefit of additional math training or alignment tuning across various model families.

1 Introduction

Entity tracking, the capacity to track how properties of discourse entities and their relationships change as a discourse unfolds, is an important ability for understanding longer contexts as well as other critical capabilities such as planning. For example, to successfully parse the following recipe, an agent needs to track what happens to the different entities, such as ingredients.

- (1) Put the eggs, sugar, flour, and baking powder in a bowl and mix to form a light batter. Make sure that the final batter does not contain any lumps of flour or sugar.

Kim & Schuster (2023) showed that several Transformer-based large language models (LLMs), such as GPT-3.5, exhibit a non-trivial entity tracking capacity. At the same time, they found that similar models, such as GPT-3, seem to lack this ability. Based on the limited information available about the differences between the GPT-3 and GPT-3.5 models, Kim & Schuster (2023) hypothesized that pretraining on large amounts of code imbues LLMs with entity tracking abilities. However, due to the opacity of training data specifications of these models, it remains unclear whether code pretraining indeed is the critical difference. In this work, we re-evaluate this claim with open-source LLMs for which more information about the pretraining process is available. We extend our analysis to the effect of math and instruction tuning as well as code. Upon comparing pairs of base models and models additionally trained on code, math, or alignment tuned,¹ we find a clear benefit of code training but no consistent benefit of math training or alignment tuning.

*Equal contribution.

¹We use the term alignment tuning to refer to various methods of making language models more useful for interactive settings, including supervised instruction finetuning (SFT) (Wei et al., 2022a), reinforcement learning from human feedback (RLHF) (Ouyang et al., 2022), and direct preference optimization (DPO) (Rafailov et al., 2023).

Exp	Model	Size	Additional Training			Base Model
			+Code	+Math	+Instruct/Chat	
Exp 1 (code)	Llama 2	7B–70B				-
	Code Llama	7B–70B	✓			Llama 2
	DeepSeek	7B				-
	DeepSeek-Coder	7B	✓			DeepSeek
	Gemma	8B				-
	CodeGemma	8B	✓			Gemma
Exp 2 (math)	Llama	7B				-
	FLoat	7B		✓ (instruct)		Llama
	Mistral	7B				-
	OpenMathMistral	7B		✓ (instruct)		Mistral
	DeepSeek-Coder	7B	✓			DeepSeek
	DeepSeek-Math	7B	✓	✓		DeepSeek-Coder
	Code Llama	7B, 34B	✓			Llama 2
	Llemma	7B, 34B	✓	✓		Code Llama
Exp 3 (alignment)	Llama 2	7B–70B				-
	Llama 2-Chat	7B–70B			✓	Llama 2
	Code Llama	7B–70B	✓			Llama 2
	Code Llama-Instruct	7B–70B	✓		✓	Code Llama 2
	Gemma	8B				-
	Gemma-Instruct	8B			✓	Gemma
	CodeGemma	8B	✓			Gemma
	CodeGemma-Instruct	8B	✓		✓	CodeGemma
	DeepSeek	7B				-
	DeepSeek-Chat	7B			✓	DeepSeek
	DeepSeek-Coder	7B	✓			DeepSeek
	DeepSeek-Coder-Instruct	7B	✓		✓	DeepSeek-Coder

Table 1: Summary of the models compared and their pretraining data composition.

2 Related work

Including code in the pretraining data mixture, even for models not explicitly specialized for code, has become increasingly customary in LLM training (Chowdhery et al., 2023; Touvron et al., 2023b; Gemini Team et al., 2024; Groeneveld et al., 2024, *i.a.*). In addition to serving the popular use case of LLMs in code completion and generation (Chen et al., 2021), adding code to the pretraining data mixture has been claimed to improve general reasoning capacities of LLMs (Fu et al., 2022; Ma et al., 2024; Yang et al., 2024). Kim & Schuster (2023) hypothesized that a concrete capacity that can benefit from code is entity tracking: converging evidence towards this claim is contributed by observations from Madaan et al. (2022) (code pretrained models like Codex perform better than models primarily trained on language data on ProPara (Dalvi et al., 2019)), Sap et al. (2022) (GPT-3.5 performs better on object tracking than GPT-3), and Muennighoff et al. (2023) (adding code to the pretraining data improves performance the on bAbI tasks (Weston et al., 2016)). Furthermore, Prakash et al. (2024) observed that a base model finetuned on arithmetic tasks improved performance on a simplified version of the entity tracking task by Kim & Schuster (2023), suggesting that structured data in general beyond code may contribute to the development of an entity tracking capacity in language models.

3 Experiments

We aim to systematically test the hypothesis that code pretraining leads to better entity tracking put forward by Kim & Schuster (2023), through a series of experiments comparing base models and models continued to be trained on code on top of the base models. We additionally test the hypothesis that pretraining on math, another type of structured data, leads to better entity tracking performance through similar comparisons.

2-shot prompt

Given the description after "Description:", write a true statement about all boxes and their contents to the description after "Statement:".

Description: Box 0 contains the car, Box 1 contains the cross, Box 2 contains the bag and the machine, Box 3 contains the paper and the string, Box 4 contains the bill, Box 5 contains the apple and the cash and the glass, Box 6 contains the bottle and the map.

Statement: Box 0 contains the car, Box 1 contains the cross, Box 2 contains the bag and the machine, Box 3 contains the paper and the string, Box 4 contains the bill, Box 5 contains the apple and the cash and the glass, Box 6 contains the bottle and the map.

Description: Box 0 contains the car, Box 1 contains the cross, Box 2 contains the bag and the machine, Box 3 contains the paper and the string, Box 4 contains the bill, Box 5 contains the apple and the cash and the glass, Box 6 contains the bottle and the map. Remove the car from Box 0. Remove the paper and the string from Box 3. Put the plane into Box 0. Move the map from Box 6 to Box 2. Remove the bill from Box 4. Put the coat into Box 3.

Statement: Box 0 contains the plane, Box 1 contains the cross, Box 2 contains the bag and the machine and the map, Box 3 contains the coat, Box 4 contains nothing, Box 5 contains the apple and the cash and the glass, Box 6 contains the bottle.

Description: {description}

Statement: Box 0 contains

Table 2: Prompts with 2-shot in-context demonstrations.

3.1 Models

We selected model pairs that have been reported to vary only in terms of their pretraining data. For testing the code hypothesis, we compared the following pairs of models: (Llama 2, Code Llama), (DeepSeek, DeepSeek-Coder), and (Gemma, CodeGemma), where the second model in each pair is obtained by continuing to train the first model on additional code data. We tested 7B, 13B, and 70B models in the Llama 2 series. For testing the math hypothesis, we compared the following four pairs of models: (Code Llama, Llemma), (DeepSeek-Coder, DeepSeek-Math), (Llama, FLoat), and (Mistral, OpenMathMistral). Again, the second model in each pair is obtained by training the first model on additional math data. For alignment tuning, we compared (Llama 2, Llama 2-chat), (Code Llama, Code Llama-Instruct), (Gemma, Gemma-Instruct), (CodeGemma, CodeGemma-Instruct), (DeepSeek, DeepSeek-Chat), and (DeepSeek-Coder, DeepSeek-Coder-Instruct). These comparisons are summarized in Table 1. See Appendix A.1, Table 4 for more details about the models.

3.2 Evaluation setup

We adopted the boxes task (the “base” version) from Kim & Schuster (2023) for testing the models’ entity tracking capacity. In this task, the input to the LLM is a textual description of the contents of seven boxes followed by 1–12 descriptions of operations that change the contents of the individual boxes. In response to this input, the LLM is prompted to state the contents of each box according to the initial description and the state-changing operations. We used the same prompt and 2-shot in-context learning examples as Kim & Schuster (2023) (see Table 2 for an example). We used a slightly different prompt format for chat-optimized models to align the task better to the input format the models were trained on. The inputs to the model are provided as “user” prompts, and the expected model outputs are formatted as “assistant” (see Table 5 in Appendix for an example prompt).